



School of Science & Technology

IN1011 Operating Systems Stage 1 **Mock Examination A**

May/June

Examination duration: **90** minutes

Division of marks: Answer **3** of the **4** questions (20 marks for each question). If more than **3** questions are answered, the best **3** will be counted. Total mark is out of **60**

Other instructions: All necessary working must be shown.

BEGIN EACH QUESTION ON A FRESH PAGE

Number of answer books to be provided: 1

Calculators permitted: Yes

Dictionaries permitted: None

Can question paper be removed from the examination room: No

Additional materials: **None**

Question 1

- a. Program A() is executed twice on a system. The executions are not concurrent. The global variable *flag* is initialised to **true** for one of the executions, and **false** for the other execution.

```
A() {  
    while(flag) {  
        int n = 1;  
        flag = false;  
    }  
}
```

- i. Each execution will have an associated process image. Give 2 similarities and 2 differences between the process images. **[4 Marks]**
 - ii. What does it mean for a process to be *CPU-bound*? Is A() CPU-bound? Why? **[3 Marks]**
- b. The OS uses control stacks when interrupts occur.
- i. What is the purpose of a control stack when the OS handles an interrupt? Your answer should include 2 examples of data copied to the stack. **[3 Marks]**
 - ii. What property shared by all stacks makes a control stack very useful to the OS when nested interrupts occur? Your answer should name the property and briefly explain why this property is useful for nested routines. **[3 Marks]**
- c. A common technique for achieving mutual exclusion on a uniprocessor system is disabling interrupts before entering a critical section and enabling interrupts after exiting a critical section.
- i. Why does this work? **[2 Marks]**
 - ii. Why won't this work on a multiprocessor system? **[2 Marks]**
- d. A typical hardware architecture provides an instruction called *return from interrupt* (RETI). This instruction switches the CPU mode of operation from kernel to user mode.
- i. Which OS routines would invoke this instruction? **[1 Mark]**
 - ii. Speculate on what happens if a user program invokes this instruction. **[2 Marks]**

Question 2

- a. Upon creation, 3 processes (A, B and C) are admitted into the ready queue in quick succession – i.e., one immediately after another. You notice the processes always successfully terminate in the order A, B, C – and the average turnaround time is *always the same* – irrespective of the order in which the processes are first admitted into the ready queue. The processes make no IO requests and have different service times.
- i. What scheduling policy could the OS be employing? Explain why you think this.
[3 Marks]
 - ii. Name 2 scheduling policies that are not employed by the OS. Explain why you think this for each policy you mention.
[4 Marks]
- b. Suppose the 3 processes in question 3b always successfully terminate in the order that they are first admitted to the ready queue. And, that the average turnaround time is *dependent* on the order of first admittance to the ready queue.
- i. What scheduling policy could the OS be employing? Explain why you think this.
[3 Marks]
 - ii. Name 3 scheduling policies that are not employed by the OS. Explain why you think this for each policy you mention.
[6 Marks]
- c. Finally, suppose the 3 processes in question 3b always terminate in the reverse order in which they are first admitted to the ready queue (you may ignore turnaround times). Briefly describe a scheduling policy that ensures this happens. Your description should clearly state the *selection function* and *decision mode* for the policy.
[4 Marks]

Question 3

- a. In the context of demand-paging, what does TLB stand for? Explain why the TLB is necessary to achieve acceptable performance.

[5 Marks]

- b. Give a formula for EAT in terms of a (the base access time), p (the page-fault rate) and S (the average page-fault service time).

[4 Marks]

- c. In a virtual memory system based on paging, what is a page table? Draw a diagram to show how a page table (without any translation look-aside buffer) is used to translate a virtual memory address to a physical memory address.

[6 Marks]

- d. Calculate the Effective Access Time and Slowdown rate with the following conditions:
- i. base memory access time = 200ns and average page-fault service time = 0.1ms.
 - ii. suppose one memory reference in every 10000 results in a page-fault.

[5 Marks]

Question 3

- a. Consider a swapping system that has a series of holes (all in MB) with the following sizes (in order): **9, 20, 15, 22, 18, 17, 19, 31, 11, 14**

Three processes of sizes **9, 11, 13** MB arrive in order. Select the correct order of the holes selected if a best fit strategy is used.

[5 Marks]

- b. Consider the following scenario. There are two users, user one is called troy and belongs to a group called troy. The second user is called zapata and is NOT in the group troy. Then, user troy runs two ls commands in his home directory /home/troy, running a bash shell, with the following results:

```
troy@debian:~$ ls -al
total 12
drwxr-xr-x  3 troy troy 4096 Mar 20 22:12 .
drwxr-xr-x 22 troy troy 4096 Mar 20 22:12 ..
drwxr-x--x  2 troy troy 4096 Mar 20 21:35 poems

troy@debian:~$ ls -l poems
total 8
-rw-r--rw-  1 troy troy 1030 Mar 20 21:35 jabberwocky
-rw-r-----  1 troy troy 3427 Mar 20 21:33 jumblies
```

Now, user zapata changes directory into /home/troy and enters the following sequence of commands also running a bash shell. Describe and explain the output obtained in each case. Remember, zapata is not a member of group troy, and assume that jumblies and jabberwocky are both non-empty plain text files.

- i. zapata@debian:/home/troy\$ ls ---al **[3 Marks]**
- ii. zapata@debian:/home/troy\$ ls -l poems **[3 Marks]**
- iii. zapata@debian:/home/troy\$ ls -l poems/jumblies **[3 Marks]**
- iv. zapata@debian:/home/troy\$ cat poems/jumblies **[3 Marks]**
- v. zapata@debian:/home/troy\$ cat poems/jabberwocky **[3 Marks]**

End of Exam